

Aula 06 — Spring Framework

A classe do meio ù Empréstimo ù Venda

Mauricio Duailibi Neto

Turma Java Entra21

Como será a aula de hoje

Parte 1 — 18h30 às 20h

- Revisão da Aula 05
- Quando 1:N não chega
- A classe do meio
- Projeto **biblioteca** — fazemos juntos

Parte 2 — 20h20 às 22h

- Voltamos ao **StockSales**
- Venda de um produto
- Venda com vários produtos

Dois projetos nesta aula

Não vamos misturar os códigos.

biblioteca Primeira parte da aula. Livro e leitor já existem. Nós criamos o empréstimo.

StockSales O sistema da loja. Cliente e produto já existem. Só depois do intervalo.

O JavaScript das telas já está pronto.

Hoje a gente escreve o Java.

Mapa da aula

- 1 Revisão da Aula 05
- 2 A classe do meio
- 3 Projeto biblioteca
- 4 Vários livros
- 5 Parte 2 — StockSales
- 6 Recapitulando

O que já sabemos

Na aula passada um cadastro passou a apontar para outro.

- laboratório: o material guarda `categoriaId`
- Uma categoria, vários materiais — isso é 1:N
- As listas ficam na classe `Dados`
- `StockSales`: produto e cliente, cada um na sua lista

Cliente e produto ainda não se encontram.

Ainda não existe venda.

Como gravamos o 1:N

Não copiamos a categoria inteira dentro do material.

Guardamos só o número:

Material	quantidade	categoriald
Caneta	10	1
Caderno	5	1
Mouse	3	2

A categoria 1 é Papelaria.

Se o nome da categoria mudar, o material continua ligado ao id 1.

Os quatro verbos continuam iguais

Ação	Verbo HTTP	O que faz
Listar / ler	GET	Lê, não muda nada
Cadastrar	POST	Cria um item novo
Editar	PUT	Troca os dados de um item
Excluir	DELETE	Apaga um item

Hoje o POST ganha um trabalho extra:
achar os dois cadastros e conferir a quantidade.

O problema de hoje

Na biblioteca, Maria quer levar Dom Casmurro.

Não dá para colocar o livro dentro do leitor.

O leitor levaria um livro só, para sempre.

Não dá para colocar o leitor dentro do livro.

O livro serviria para uma pessoa só.

Pergunta de hoje

Onde gravamos o encontro dos dois,
com a quantidade que saiu da prateleira?

Por que um id só não resolve

No 1:N, o material aponta para **uma** categoria.

O empréstimo aponta para **dois** lados:

- quem leva — o leitor
- o que leva — o livro

E ainda precisa de coisa própria:

- um id
- a quantidade

Isso não cabe em `leitorId` dentro do livro.

Nem em `livroId` dentro do leitor.

A classe do meio

Criamos um cadastro novo.

Cada linha é um encontro.

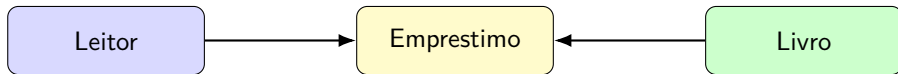
id	leitorId	livroId	quantidade
1	1 Maria	1 Dom Casmurro	1
2	3 Helena	3 Capitaes da Areia	2

Maria continua na lista de leitores.

O livro continua na lista de livros.

O empréstimo só guarda os ids e a quantidade.

O desenho



O amarelo é a classe do meio.

Sem ela, leitor e livro não se encontram.

Um leitor tem vários empréstimos.

Um livro aparece em vários empréstimos.

Cada empréstimo, neste primeiro passo, aponta para **um** livro.

Dois passos na aula

Passo	O que a classe do meio guarda
1	Um leitor e um livro, com quantidade
2	Um leitor e uma lista de livros

Primeiro a gente faz o passo 1 funcionar na tela.
Só depois a lista.

No StockSales é a mesma ordem:
um produto, depois vários produtos.

O que não vamos usar hoje

- Banco de dados
- Anotações do tipo `@ManyToMany`
- JPA
- Data de empréstimo

Continuamos com lista na memória e um `int` guardando o id do outro cadastro.

O raciocínio é o mesmo que o banco vai usar depois.
Só a ferramenta muda.

O projeto biblioteca

Projeto novo. Não é o laboratório.

O que já está pronto

Livro e leitor com GET, POST, PUT e DELETE.

Telas com formulário. Axios já chama a API de empréstimo.

Se a API ainda não existe, a tela avisa.

Quando o Java aparecer, a lista responde.

Passo 1 — Subir o Spring

Onde estamos

Projeto **biblioteca**

No terminal, dentro da pasta biblioteca:

```
./mvnw spring-boot:run
```

No Windows: `.\mvnw.cmd spring-boot:run`

Abram no navegador: `http://localhost:8080`

Um Spring por vez

Se outro projeto ainda estiver na porta 8080, parem o outro antes.

O que vocês vão ver na tela

- Livros: Dom Casmurro, O Cortico, Capitaes da Areia, A Hora da Estrela
- Leitores: Maria, Jose, Helena
- Novo empréstimo: dois selects e a quantidade
- Histórico: lista vazia, por enquanto

Cadastrar livro e leitor já funciona.

Confirmar empréstimo ainda não.

Esse Java a gente escreve agora.

Onde cada arquivo fica

```
biblioteca/  
  src/main/java/com/entra21/biblioteca/  
    BibliotecaApplication.java  
    Dados.java  
    model/Livro.java  
    model/Leitor.java  
    model/Emprestimo.java      novo  
    controller/LivroController.java  
    controller/LeitorController.java  
    controller/EmprestimoController.java  novo  
  src/main/resources/static/  
    emprestimos-novo.html  
    emprestimos.html
```

Onde estamos

Projeto **biblioteca**

Arquivo Dados.java, acrescentar

```
public static ArrayList<Emprestimo> empréstimos =  
    new ArrayList<>();  
public static int proximoIdEmprestimo = 1;
```

Import de Emprestimo no topo do arquivo.

Sem isso, o Controller não tem onde guardar.

Classe Emprestimo

Onde estamos

Arquivo novo:

src/main/java/com/entra21/biblioteca/model/Emprestimo.java

```
public class Emprestimo {  
    private int id;  
    private int leitorId;  
    private int livroId;  
    private int quantidade;  
  
    public Emprestimo() {  
    }  
}
```

Construtor vazio e os gets e sets dos quatro atributos.
O vazio é obrigatório no POST.

O JSON que a tela já manda

Vocês não mexem no JavaScript.

A página já envia isto:

```
{"leitorId": 1, "livroId": 2, "quantidade": 1}
```

O servidor faz o resto:

- 1 Achar o leitor
- 2 Achar o livro
- 3 Se a quantidade do livro for menor, recusar
- 4 Diminuir a quantidade do livro
- 5 Guardar o empréstimo

Uma função só para achar

Onde estamos

controller/EmprestimoController.java, arquivo novo

Em vez de um for gigante dentro do POST, uma função pequena devolve o livro ou null.

```
public Livro acharLivro(int id) {  
    for (int i = 0; i < Dados.livros.size(); i++) {  
        if (Dados.livros.get(i).getId() == id) {  
            return Dados.livros.get(i);  
        }  
    }  
    return null;  
}
```

Façam a mesma ideia para acharLeitor.

GET dos empréstimos

Onde estamos

Ainda em `EmprestimoController.java`

```
@RestController
public class EmprestimoController {

    @GetMapping("/api/emprestimos")
    public ArrayList<Emprestimo> listar() {
        return Dados.emprestimos;
    }
}
```

Testem no navegador:

`http://localhost:8080/api/emprestimos`

Deve aparecer []. Lista vazia, mas a rota existe.

POST — achar leitor e livro

Onde estamos

Ainda no método cadastrar

```
@PostMapping("/api/emprestimos")
public Emprestimo cadastrar(@RequestBody Emprestimo pedido) {
    Leitor leitor = acharLeitor(pedido.getLeitorId());
    if (leitor == null) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Leitor nao encontrado");
    }

    Livro livro = acharLivro(pedido.getLivroId());
    if (livro == null) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Livro nao encontrado");
    }
}
```

O método ainda não acabou.

POST — quantidade e gravação

Onde estamos

Ainda no mesmo método cadastrar

```
if (livro.getQuantidade() < pedido.getQuantidade()) {  
    throw new ResponseStatusException(  
        HttpStatus.BAD_REQUEST, "Quantidade insuficiente");  
}  
  
pedido.setId(Dados.proximoIdEmprestimo);  
Dados.proximoIdEmprestimo = Dados.proximoIdEmprestimo + 1;  
livro.setQuantidade(  
    livro.getQuantidade() - pedido.getQuantidade());  
Dados.emprestimos.add(pedido);  
return pedido;  
}
```


Imports deste Controller

- Dados, Emprestimo, Leitor, Livro
- HttpStatus
- ResponseStatusException
- ArrayList
- As anotações GetMapping, PostMapping, RequestBody, RestController

Se faltar um import, o VS Code sublinha em vermelho.

Aceitem o import que ele oferece.

Conferindo o passo 1

`/api/emprestimos` devolve []

Maria leva 1 Dom Casmurro — a tela diz registrado

Em Livros, a quantidade do titulo caiu

O histórico mostra o nome, não só o id

Pedir mais exemplares do que existe — a tela recusa

Se a quantidade não cair, o `setQuantidade` não rodou.

Se o histórico continuar vazio, o GET não devolveu a lista.

Erros que mais aparecem

- 404 no POST — o Controller não compilou ou o Spring não reiniciou
- Lista de livros some — o seed não está no if (`Dados.livros.isEmpty()`)
- Quantidade não diminui — acharam o livro e não gravaram de volta no objeto
- Porta 8080 ocupada

Um empréstimo, vários livros

Na vida real a pessoa sai com mais de um título.

Um campo `livroId` não basta.

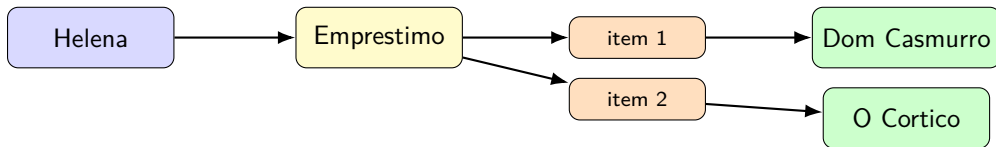
Precisamos de uma lista dentro do empréstimo.

Cada linha da lista é um **item**:
este livro, esta quantidade.

O POST de um livro **continua no ar**.

Vamos acrescentar outro endereço.

O desenho com lista



Um empréstimo. Dois itens. Dois livros.

Classe ItemEmprestimo

Onde estamos

Arquivo novo:

model/ItemEmprestimo.java

```
public class ItemEmprestimo {  
    private int livroId;  
    private int quantidade;  
  
    public ItemEmprestimo() {  
    }  
}
```

Gets e sets dos dois atributos.

Esta classe não precisa de id próprio nesta aula.

Lista de itens no Emprestimo

Onde estamos

model/Emprestimo.java, acrescentar

```
private ArrayList<ItemEmprestimo> itens;  
  
public ArrayList<ItemEmprestimo> getItens() {  
    return itens;  
}  
  
public void setItens(ArrayList<ItemEmprestimo> itens) {  
    this.itens = itens;  
}
```

Os campos livroId e quantidade ficam.
O POST de um livro ainda usa eles.

O JSON da tela de vários livros

A página `emprestimos-varios.html` já manda:

```
{
  "leitorId": 3,
  "itens": [
    {"livroId": 1, "quantidade": 1},
    {"livroId": 2, "quantidade": 1}
  ]
}
```

Endereço: `POST /api/emprestimos/itens`

Dois fors, nessa ordem

Primeiro conferimos **tudo**.

Só depois baixamos quantidade.

Se o segundo livro não tiver exemplar,
o primeiro também não sai.

- 1 for nos itens: achar o livro, conferir quantidade
- 2 Se algum falhar, recusar o empréstimo inteiro
- 3 Outro for: diminuir a quantidade de cada livro
- 4 Gravar o empréstimo

POST de vários — conferir

Onde estamos

EmprestimoController.java, método novo

```
@PostMapping("/api/emprestimos/itens")
public Emprestimo cadastrarVarios(@RequestBody Emprestimo pedido) {
    Leitor leitor = acharLeitor(pedido.getLeitorId());
    if (leitor == null) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Leitor nao encontrado");
    }

    for (int i = 0; i < pedido.getItens().size(); i++) {
        ItemEmprestimo item = pedido.getItens().get(i);
        Livro livro = acharLivro(item.getLivroId());
        if (livro == null) {
            throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST, "Livro nao encontrado");
        }
    }
}
```

POST de vários — gravar

Onde estamos

Ainda no mesmo método

```
    if (livro.getQuantidade() < item.getQuantidade()) {  
        throw new ResponseStatusException(  
            HttpStatus.BAD_REQUEST, "Quantidade insuficiente");  
    }  
}  
  
for (int i = 0; i < pedido.getItems().size(); i++) {  
    ItemEmprestimo item = pedido.getItems().get(i);  
    Livro livro = acharLivro(item.getLivroId());  
    livro.setQuantidade(  
        livro.getQuantidade() - item.getQuantidade());  
}  
  
pedido.setId(Dados.proximoIdEmprestimo);  
Dados.proximoIdEmprestimo = Dados.proximoIdEmprestimo + 1;  
Dados.emprestimos.add(pedido);  
return pedido;  
}
```

Conferindo o passo 2

- A tela Vários livros deixa acrescentar dois títulos
- Confirmar registra e as duas quantidades caem
- O histórico mostra os dois nomes
- O empréstimo de um livro ainda funciona

Se não der tempo agora, a gente faz a mesma lista no StockSales, depois do intervalo.

Checklist da Parte 1

Estou no projeto biblioteca?

Dados tem a lista de empréstimos?

/api/emprestimos existe?

A quantidade do livro diminui?

A tela recusa quando não há exemplar?

Retomamos às **20h20**

Fechem a biblioteca.

Parem o Spring.

Na segunda parte abrimos o **StockSales**.

Mudamos de projeto

Onde estamos

Agora é o projeto **StockSales**.
A biblioteca já está encerrada.

Parem o Spring da biblioteca.
Abram a pasta do StockSales e subam de novo.

Onde o StockSales está

A loja já tem produto e cliente.

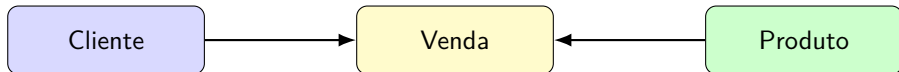
- Listas em Dados
- Telas de cadastro
- Telas de venda já prontas, com Axios

Ainda **não** tem a classe Venda.

Ainda **não** tem o Controller de venda.

É o empréstimo, com outro nome, e com preço.

O desenho da venda de hoje



Um cliente tem várias vendas.

Um produto aparece em várias vendas.

Cada venda, neste primeiro passo, aponta para **um** produto.

O que a venda tem a mais

Igual ao empréstimo:

- `clienteId`
- `produtoId`
- `quantidade`

Extra da loja:

- `total` — preço vezes quantidade
- o estoque do produto diminui

O JavaScript **não** calcula o total.

Quem calcula é o servidor.

Onde estamos

Projeto **StockSales**

Arquivo Dados.java, acrescentar

```
public static ArrayList<Venda> vendas = new ArrayList<>();  
public static int proximoIdVenda = 1;
```

Import de Venda no topo.

Classe Venda

Onde estamos

Arquivo novo:

src/main/java/com/entra21/stocksales/model/Venda.java

```
public class Venda {  
    private int id;  
    private int clienteId;  
    private int produtoId;  
    private int quantidade;  
    private double total;  
  
    public Venda() {  
    }  
}
```

Construtor vazio e os gets e sets dos cinco atributos.

O JSON da nova venda

A página vendas-nova.html já manda:

```
{"clienteId": 1, "produtoId": 2, "quantidade": 3}
```

O servidor:

- 1 Acha cliente e produto
- 2 Se estoque < quantidade, recusa
- 3 $\text{total} = \text{preco} * \text{quantidade}$
- 4 Diminui o estoque
- 5 Guarda a venda

acharCliente e acharProduto

Onde estamos

controller/VendaController.java, arquivo novo

Copiem a ideia da biblioteca.

```
public Produto acharProduto(int id) {  
    for (int i = 0; i < Dados.produtos.size(); i++) {  
        if (Dados.produtos.get(i).getId() == id) {  
            return Dados.produtos.get(i);  
        }  
    }  
    return null;  
}
```

A função acharCliente é a mesma, na lista Dados.clientes.

GET e começo do POST

Onde estamos

Ainda em VendaController.java

```
@GetMapping("/api/vendas")
public ArrayList<Venda> listar() {
    return Dados.vendas;
}

@PostMapping("/api/vendas")
public Venda cadastrar(@RequestBody Venda pedido) {
    Cliente cliente = acharCliente(pedido.getClienteId());
    if (cliente == null) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Cliente nao encontrado");
    }
}
```

POST — estoque, total, gravação

Onde estamos

Ainda no método cadastrar

```
Produto produto = acharProduto(pedido.getProdutoId());
if (produto == null) {
    throw new ResponseStatusException(
        HttpStatus.BAD_REQUEST, "Produto nao encontrado");
}

if (produto.getEstoque() < pedido.getQuantidade()) {
    throw new ResponseStatusException(
        HttpStatus.BAD_REQUEST, "Estoque insuficiente");
}

pedido.setId(Dados.proximoIdVenda);
Dados.proximoIdVenda = Dados.proximoIdVenda + 1;
pedido.setTotal(produto.getPreco() * pedido.getQuantidade());
produto.setEstoque(
    produto.getEstoque() - pedido.getQuantidade());
Dados.vendas.add(pedido);
return pedido;
}
```


O caminho da primeira venda

- 1 A pessoa escolhe Ana, Mouse Gamer, quantidade 2
- 2 JS manda POST /api/vendas
- 3 Servidor acha cliente e produto em Dados
- 4 Confere estoque, calcula total, diminui estoque
- 5 A tela de produtos, ao recarregar, já mostra o estoque menor

Conferindo a venda de um produto

Nova venda baixa o estoque

O histórico mostra o nome e o total

Vender acima do estoque mostra a mensagem

O total não vem digitado na tela

Venda com vários produtos

Mesma evolução da biblioteca.

- Classe ItemVenda: `produtoId`, `quantidade`, `subtotal`
- Campo `itens` na Venda
- POST `/api/vendas/itens`
- A tela `vendas-varios.html` já existe

O POST de um produto permanece.

Classe ItemVenda

Onde estamos

Arquivo novo: model/ItemVenda.java

```
public class ItemVenda {  
    private int produtoId;  
    private int quantidade;  
    private double subtotal;  
  
    public ItemVenda() {  
    }  
}
```

Gets e sets dos três atributos.

O subtotal a tela não manda. O servidor preenche.

O JSON de vários produtos

```
{
  "clienteId": 1,
  "itens": [
    {"produtoId": 2, "quantidade": 1},
    {"produtoId": 4, "quantidade": 2}
  ]
}
```

Primeiro for: conferir estoque de cada item.

Segundo for: subtotal, somar o total, baixar estoque.

Somar o total

Onde estamos

No segundo for de cadastrarVarios

```
double total = 0;
for (int i = 0; i < pedido.getItems().size(); i++) {
    ItemVenda item = pedido.getItems().get(i);
    Produto produto = acharProduto(item.getProdutoId());
    item.setSubtotal(produto.getPreco() * item.getQuantidade());
    total = total + item.getSubtotal();
    produto.setEstoque(
        produto.getEstoque() - item.getQuantidade());
}
pedido.setTotal(total);
```

total = total + ... vai juntando cada subtotal.

Conferindo vários produtos

Ana leva Mouse e Headset na mesma venda

Os dois estoques caem

O total é a soma dos dois subtotais

Se um produto não tem estoque, a venda inteira é recusada

A venda de um produto continua funcionando

O que não entra hoje

- Cancelar venda e devolver estoque
- Banco de dados
- Login
- Anotações JPA

Meta no StockSales

Venda de um item, com estoque baixando e total no servidor.
Se der tempo, vários itens na mesma venda.

Venda grava `clienteId` e `produtoId`

Total é calculado no servidor

Estoque diminui

Estoque insuficiente recusa a venda

Histórico mostra o **nome**, não só o id

O que vimos hoje

- ① 1:N guarda um id no lado dos muitos
- ② Quando dois cadastros se encontram, nasce uma classe nova
- ③ Essa classe guarda os dois ids e a quantidade
- ④ Uma lista de itens deixa vários produtos na mesma venda
- ⑤ Conferir tudo antes de baixar estoque

Os dois desenhos, lado a lado

biblioteca	StockSales
Leitor e Livro	Cliente e Produto
Emprestimo no meio	Venda no meio
quantidade do livro cai	estoque do produto cai
lista de itens	lista de itens, com total

Mesma ideia. Nomes diferentes.

Aula 07

- Validação dos dados que chegam
- Mensagem de erro mais clara
- Organizar o que já cresceu no StockSales

Tragam o StockSales com a venda de um item funcionando.

Obrigado!

Mauricio Duailibi Neto
Turma Java Entra21